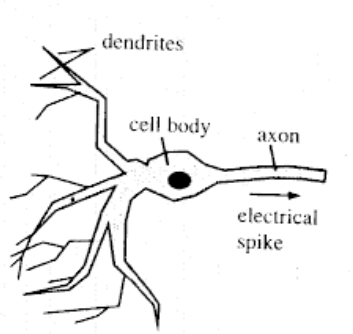
6.7 ข่ายงานประสาทเทียม

ข่ายงานประสาทเทียม (Artificial Neural Network) เป็นการจำลองการทำงานบางส่วนของสมองมนุษย์ เซลล์ประสาท (neuron) ในสมองของคนเราประกอบด้วยนิวเคลียส (nucleus) ตัวเซลล์ (cell body) ใยประสาทนำเข้า (dendrite) แกนประสาทนำออก (axon) แสดงในรูปที่ 6–34



รูปที่ 6–34 เซลล์ประสาท

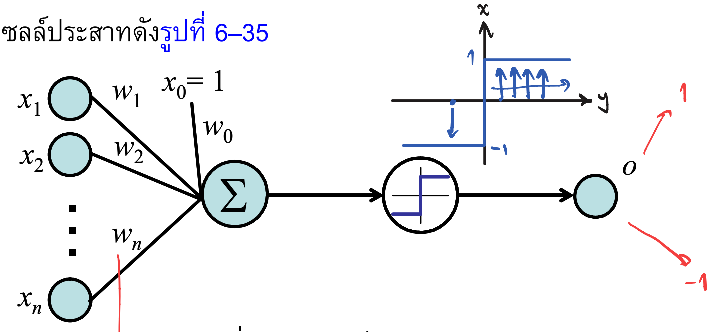
เดนไดรท์ทำหน้าที่รับสัญญาณไฟฟ้าเคมีซึ่งส่งมาจากเซลล์ประสาทใกล้เคียง เซลล์ประสาทตัวหนึ่งๆ จะเชื่อมต่อกับเซลล์ตัวอื่นๆ ประมาณ 10,000 ตัว เมื่อสัญญาณไฟฟ้าเคมีที่รับเข้ามาเกินค่าค่าหนึ่ง เซลล์จะถูกกระตุ้นและส่งสัญญาณไปทางแกนประสาทนำออกไปยังเซลล์อื่นๆ ต่อไป ประมาณกันว่าสมองของคนเรามีเซลล์ประสาทอยู่ทั้งสิ้นประมาณ 10^{11} ตัว

6.7.1 เพอร์เซปตรอน

เพอร์เซปตรอน (perceptron) เป็นข่ายงานประสาทเทียมแบบง่ายมีหน่วยเดียวที่จำลองลักษณะของเซลล์ประสาทดังรูปที่ 6–35

เพิ่มมาเพื่อถ่วงน้ำหนัก

$\boxed{x_0w_0} + x_1w_1 + x_2w_2 + x_3+w_3+ \ldots + x_nw_n$

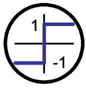


รูปที่ 6–35 เปอร์เซปตรอน

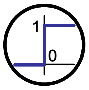
สามารถปรับ w (น้ำหนัก) ได้

เพอร์เซปตรอนรับอินพุตเป็นเวกเตอร์จำนวนจริงแล้วคำนวณหาผลรวมเชิงเส้น (linear combination) แบบถ่วงน้ำหนักของอินพุต (x_1, x_2, ..., x_n) โดยที่ค่า w_1, w_2, ..., w_n ในรูปเป็นค่าน้ำหนักของอินพุตและให้เอาต์พุต (o) เป็น 1 ถ้าผลรวมที่ได้มีค่าเกินค่าขีดแบ่ง (θ) และเป็น -1 ถ้าไม่เกิน ส่วน w_0 ในรูปเป็นค่าลบของค่าขีดแบ่งดังจะได้อธิบายต่อไป และ x_0 เป็นอินพุตเทียมกำหนดให้มีค่าเป็น 1 เสมอ

ฟังก์ชันกระตุ้น



ในรูปแสดง*ฟังก์ชันกระตุ้น (activation function)* ชนิดที่เรียกว่า*ฟังก์ชันสองขั้ว (bipolar function)* ซึ่งแสดงผลของเอาต์พุตเป็น 1 กับ -1 ฟังก์ชันกระตุ้นอื่นๆ ที่นิยมใช้ก็อย่างเช่น *ฟังก์ชันไบนารี (binary function)* ซึ่งแสดงผลของเอาต์พุตเป็น 1 กับ 0 และเขียนแทนด้วยรูป



เราสามารถแสดงเอาต์พุต (o) ในรูปของฟังก์ชันของอินพุต (x_1, x_2, ..., x_n) ได้ดังนี้

$$o(x_1, x_2, \ldots, x_n) = \begin{cases} 1 \;\; if \;\; w_1x_1 + w_2x_2 + \cdots + w_nx_n \ge \boxed{\theta} \leftarrow \text{Threshold} \\ -1 \;\; if \;\; w_1x_1 + w_2x_2 + \cdots + w_nx_n < \boxed{\theta} \end{cases} \tag{6.7}$$

เอาต์พุตเป็นฟังก์ชันของอินพุตในรูปของผลรวมเชิงเส้นแบบถ่วงน้ำหนัก น้ำหนักจะเป็นตัวกำหนดว่าในจำนวนอินพุตนั้น อินพุต (x_i) ตัวใดมีความสำคัญต่อการกำหนดค่าเอาต์พุต ตัวที่มีความสำคัญมากจะมีค่าสัมบูรณ์ของน้ำหนักมาก ส่วนตัวที่มีความสำคัญน้อยจะมีค่าใกล้ศูนย์ ในกรณีที่ผลรวมเท่ากับค่าขีดแบ่งค่าเอาต์พุตไม่นิยาม (จะเป็น 1 หรือ -1 ก็ได้)

จากฟังก์ชันในสูตรที่ (6.7) เราจัดรูปใหม่โดยย้าย θ ไปรวมกับผลรวมเชิงเส้นแล้วแทน $-\theta$ ด้วย w_0 เราจะได้ฟังก์ชันของเอาต์พุตดังด้านล่างนี้

$$o(x_1, x_2, \ldots, x_n) = \begin{cases} 1 \;\; if \;\; w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n > 0 \\ -1 \;\; if \;\; w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n < 0 \end{cases} \tag{6.8}$$

กำหนดให้ $g(\vec{x}) = \sum_{i=0}^{n} w_ix_i = \vec{w} \cdot \vec{x}$ โดยที่ $\vec{x}$ แทนเวกเตอร์อินพุต เราสามารถเขียนฟังก์ชันของเอาต์พุตได้ใหม่ดังนี้

$$o(x_1, x_2, \ldots, x_n) = \begin{cases} 1 \;\; if \;\; g(\vec{x}) > 0 \\ -1 \;\; if \;\; g(\vec{x}) < 0 \end{cases} \tag{6.9}$$

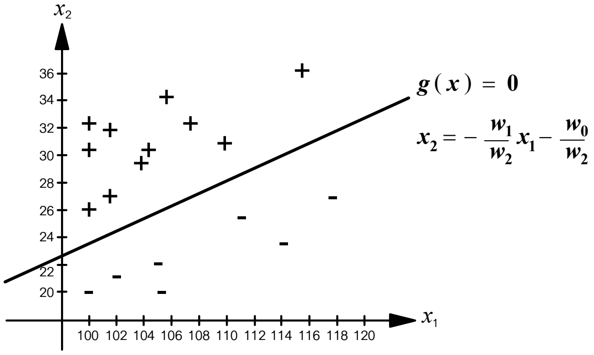
สมมติว่าเรามีอินพุตสองตัวคือ x_1 และ x_2 ซึ่งแสดงค่าส่วนสูงและน้ำหนักของเด็กนักเรียนประถมและหลังจากที่แพทย์ตรวจร่างกายของเด็กโดยละเอียดแล้วได้จำแนกนักเรียน

ออกเป็นสองกลุ่มคือเด็กอ้วนและเด็กไม่อ้วน เราให้เอาต์พุตเป็นค่าที่แสดงเด็กอ้วนแทนด้วย +1 กับไม่อ้วนแทนด้วย -1 ดังตารางที่ 6–16

ตารางที่ 6–16 ข้อมูลเด็กอ้วนและเด็กไม่อ้วน

เด็กคนที่	ส่วนสูง (ซม.)	น้ำหนัก (กก.)	อ้วน/ไม่อ้วน
1	100.0	20.0	-1
2	100.0	26.0	1
3	100.0	30.4	1
4	100.0	32.4	1
5	101.6	27.0	1
6	101.6	32.0	1
7	102.0	21.0	-1
8	103.6	29.6	1
9	104.4	30.4	1
10	104.9	22.0	-1
11	105.2	20.0	-1
12	105.6	34.4	1
13	107.2	32.4	1
14	109.9	34.9	1
15	111.0	25.4	-1
16	114.2	23.5	-1
17	115.5	36.3	1
18	117.8	26.9	-1

ในกรณีที่มีอินพุต 2 ตัว (ไม่รวม x_0) เราจะได้ $g(\vec{x}) = w_0 + w_1x_1 + w_2x_2$ ซึ่งถ้าเราให้ $g(\vec{x}) = 0$ จะได้ว่า $w_0 + w_1x_1 + w_2x_2 = 0$ ซึ่งแทนสมการเส้นตรงในระนาบสองมิติ x_1, x_2 สมการนี้มีจุดตัดแกนอยู่ที่ $-\frac{w_0}{w_2}$ และมีความชันเท่ากับ $-\frac{w_1}{w_2}$ เมื่อนำสมการนี้ไปวาดในระนาบสองมิติร่วมกับตัวอย่างสอนในตารางที่ 6–16 โดยกำหนดค่า w_0, w_1, w_2 ที่เหมาะสมจะได้ดังรูปที่ 6–36



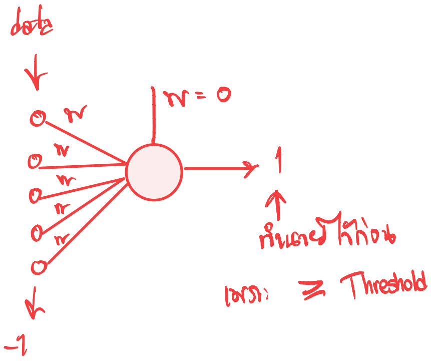
รูปที่ 6–36 สมการเส้นตรงสร้างโดยเพอร์เซบตรอน

เครื่องหมาย + และ – ในรูปแทนตัวอย่างบวก (เด็กอ้วน) และตัวอย่างลบ (เด็กไม่อ้วน) ตามลำดับ ดังจะเห็นได้ในรูปว่าเส้นตรงนี้เมื่อกำหนดจุดตัดแกนและความชันที่เหมาะสมซึ่งกำหนดโดย w_0, w_1, w_2 เส้นตรงนี้จะแบ่งตัวอย่างออกเป็นสองกลุ่มซึ่งอยู่คนละด้านของเส้นตรง และเมื่อมีข้อมูลส่วนสูงและน้ำหนักของเด็กคนอื่นที่เราต้องการทำนายว่าจะเป็นเด็กอ้วนหรือไม่ ก็ใช้เส้นตรงนี้โดยดูว่าข้อมูลใหม่นี้อยู่ด้านใดของเส้นตรง ถ้าด้านบนก็ทำนายว่าเป็นเด็กอ้วน (+) ถ้าด้านล่างก็ทำนายว่าเด็กไม่อ้วน (–)

ระนาบตัดสินใจหลายมิติ

ตัวอย่างด้านบนแสดงกรณีของอินพุตในสองมิติ จะเห็นได้ว่าเพอร์เซปตรอนจะเป็นเส้นตรง ในกรณีที่อินพุตมากกว่าสองมิติเพอร์เซปตรอนจะเป็น*ระนาบตัดสินใจหลายมิติ (hyperplane decision surface)* ปัญหาการเรียนรู้เพอร์เซปตรอนก็คือการหาค่าเวกเตอร์น้ำหนัก ($\vec{w}$) ที่เหมาะสมในการจำแนกประเภทของข้อมูลสอนเพื่อให้เพอร์เซปตรอนแสดงเอาต์พุตได้ตรงกับค่าที่สอน *กฎการเรียนรู้เพอร์เซปตรอน (perceptron learning rule)* ใช้สำหรับสอนเพอร์เซปตรอนโดยจะหาค่าเวกเตอร์น้ำหนักดังแสดงในตารางที่ 6–17

อัลกอริทึมเริ่มต้นจากสุ่มค่าเวกเตอร์น้ำหนัก ซึ่งโดยมากค่าที่สุ่มมานี้จะไม่ได้ระนาบหลายมิติที่แบ่งตัวอย่างได้ถูกต้องทุกตัวดังนั้นจึงต้องมีการแก้ไขน้ำหนักโดยเทียบเพอร์เซปตรอนกับตัวอย่างที่สอน หมายถึงว่าเมื่อเราป้อนตัวอย่างสอนเข้าไปในเพอร์เซปตรอน เราจะคำนวณค่าเอาต์พุตได้ นำค่าเอาต์พุตที่คำนวณได้โดยเพอร์เซปตรอนเทียบกับเอาต์พุตเป้าหมาย ถ้าตรงกันแสดงว่าจำแนกตัวอย่างได้ถูกต้อง ไม่ต้องปรับน้ำหนักสำหรับตัวอย่างนั้น แต่ถ้าไม่ตรงกันก็จะทำการปรับน้ำหนักตามสมการในอัลกอริทึม ส่วนอัตราการเรียนรู้เป็นตัวเลขบวกจำนวนน้อยๆ เช่น 0.01, 0.005 เป็นต้น อัตราการเรียนรู้นี้จะส่งผลต่อการลู่เข้าของเพอร์เซปตรอน ถ้าอัตราการเรียนรู้มีค่ามากเพอร์เซปตรอนก็จะเรียนรู้ได้เร็ว แต่ก็อาจเรียนรู้ไม่สำเร็จเนื่องจากการปรับค่ามีความหยาบเกินไป อัตราการเรียนรู้ที่มีค่าน้อยก็จะทำให้การปรับน้ำหนักทำได้อย่างละเอียดแต่ก็อาจเสียเวลาในการเรียนรู้นาน



ตารางที่ 6–17 อัลกอริทึมกฎการเรียนรู้เพอร์เซปตรอน

```
Algorithm: Perceptron-Learning-Rule

1. Initialize weights $w_i$ of the perceptron.
2. UNTIL the termination condition is met DO
     2.1 FOR EACH training example DO
           - Input the example and compute the output.
           - Change the weights if the output from the
             perceptron is not equal to the target
             output using the following rule.

                  $w_i \leftarrow w_i + \Delta w_i$

                  $\Delta w_i \leftarrow \alpha(t-o)x_i$

             where $t, o$ and $\alpha$ are the target output, the
             output from the perceptron and the learning
             rate, respectively.
```

ค่า x ที่มาพร้อมกับ y

Output

Learning Rate

Target (y ที่อยากได้)

ทำไปเรื่อยๆจนเจอ

Condition ที่จะหยุด

การปรับน้ำหนักตามกฎการเรียนรู้เพอร์เซปตรอนโดยใช้อัตราการเรียนรู้ที่มีค่าน้อยเพียงพอ จะได้ระนาบหลายมิติที่จะลู่เข้าสู่ระนาบหนึ่งที่สามารถแบ่งข้อมูลออกเป็นสองส่วน (ในกรณีที่ข้อมูลสามารถแบ่งได้) เพื่ออธิบายผลที่เกิดจากการปรับค่าน้ำหนัก เราจะลองพิจารณาพฤติกรรมของกฎการเรียนรู้นี้ดูว่าทำไมการปรับน้ำหนักเช่นนี้จึงลู่เข้าสู่ระนาบที่แบ่งข้อมูลได้อย่างถูกต้อง

- พิจารณากรณีแรกที่เพอร์เซปตรอนแยกตัวอย่างสอนตัวหนึ่งที่รับเข้ามาได้ถูกต้อง กรณีนี้จะพบว่า (t-o) จะมีค่าเป็น 0 ดังนั้น Δw_i ไม่เปลี่ยนแปลงเพราะ $\Delta w_i = \alpha(t-o)x_i$
- พิจารณาในกรณีที่เพอร์เซปตรอนให้เอาต์พุตเป็น –1 แต่เอาต์พุตเป้าหมายหรือค่าที่แท้จริงเท่ากับ 1 ในกรณีนี้หมายความว่าค่าที่เราต้องการคือ 1 แต่ค่าน้ำหนักไม่เหมาะสม ดังนั้นเพื่อที่จะทำให้เพอร์เซปตรอนให้เอาต์พุตเป็น 1 น้ำหนักต้องถูกปรับให้สามารถเพิ่มค่าของ $\vec{w}\cdot\vec{x}$ ในกรณีนี้หมายความว่าผลรวมเชิงเส้นน้อยเกินไปและน้อยกว่า 0 จึงได้เอาต์พุตเป็น -1 ดังนั้นสิ่งที่เราต้องการคือการเพิ่มค่าผลรวมเชิงเส้นเพราะถ้าเราเพิ่มค่าได้เรื่อยๆ จนมากกว่า 0 เพอร์เซปตรอนจะให้เอาต์พุตเป็น 1 ซึ่งตรงกับที่เราต้องการ พิจารณาดูดังต่อไปนี้ว่าการปรับค่าโดยกฎเรียนรู้ทำให้ผลรวมเชิงเส้นเพิ่มขึ้นได้อย่างไร กรณีนี้เราจะได้ว่า (t-o) เท่ากับ (1-(-1)) มีค่าเป็น 2 และลองพิจารณาค่าของอินพุต x_i แยกกรณีดังนี้

  - ○ ถ้า $x_i > 0$ จะได้ว่า Δw_i มากกว่า 0 เพราะว่า $\Delta w_i \leftarrow \alpha(t-o)x_i$ และ α มากกว่า 0, (t-o) = 2 และ $x_i > 0$ จากสมการการปรับน้ำหนัก $w_i \leftarrow w_i + \Delta w_i$ เมื่อ Δw_i มากกว่า 0 จะทำให้ w_i มีค่าเพิ่มขึ้นและ $\sum w_i x_i$ ก็จะมีค่าเพิ่มขึ้น เมื่อผลรวมมีค่ามากขึ้นแสดงว่าการปรับไปในทิศทางที่ถูกต้องคือเมื่อปรับไปจนกระทั่งได้ผลรวมมากกว่า 0 จะทำให้เพอร์เซปตรอนเอาต์พุตได้ถูกต้องยิ่งขึ้น
  - ○ ถ้า $x_i < 0$ เราจะได้ว่า $\alpha(t-o)x_i$ จะมีค่าน้อยกว่า 0 แสดงว่า w_i ตัวที่คูณกับ x_i ที่น้อยกว่า 0 จะลดลงทำให้ $\sum w_i x_i$ เพิ่มขึ้นเหมือนเดิม เพราะ x_i เป็นค่าลบและ w_i มีค่าลดลง ในที่สุดก็จะทำให้เพอร์เซปตรอนให้เอาต์พุตได้ถูกต้องยิ่งขึ้น
- ในกรณีที่เพอร์เซปตรอนให้เอาต์พุตเป็น 1 แต่เอาต์พุตเป้าหมายหรือค่าที่แท้จริงเท่ากับ -1 จะได้ว่า w_i ของ x_i ที่เป็นค่าบวกจะลดลง ส่วน w_i ของ x_i ที่เป็นค่าลบจะเพิ่มขึ้นและทำให้การปรับเป็นไปในทิศทางที่ถูกต้องเช่นเดียวกับในกรณีแรก

6.7.2 ตัวอย่างการเรียนฟังก์ชัน AND และ XOR ด้วยกฎเรียนรู้เพอร์เซปตรอน

พิจารณาตัวอย่างการเรียนรู้ของเพอร์เซปตรอนโดยจะให้เรียนรู้ฟังก์ชัน 2 ฟังก์ชัน ฟังก์ชันแรกคือฟังก์ชัน AND แสดงในตารางที่ 6–18 ในกรณีนี้เราใช้ฟังก์ชันไบนารีเป็นฟังก์ชันกระตุ้น

ตารางที่ 6–18 ฟังก์ชัน AND(x1,x2)

x_1	x_2	เอาต์พุตเป้าหมาย
0	0	0
0	1	0
1	0	0
1	1	1

ฟังก์ชัน AND ตามตารางด้านบนนี้จะให้ค่าที่เป็นจริงก็ต่อเมื่อ x1 และ x2 เป็นจริงทั้งคู่ (ดูที่สดมภ์เอาต์พุตเป้าหมาย) ผลการใช้กฎการเรียนรู้เพอร์เซปตรอนกับฟังก์ชัน AND แสดงในตารางที่ 6–19

1
↑
$((t-o)\alpha)x_0) + w_0$

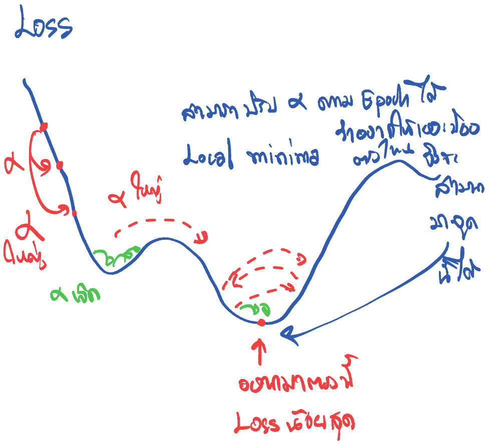
ต้องทำได้

ตารางที่ 6–19 ผลการเรียนรู้ฟังก์ชัน AND โดยกฎการเรียนรู้เพอร์เซปตรอน

1 Epoch

Perceptron Learning Example - Function AND											
		Bias Input x0=+1					Alpha =	0.5			
Input	Input				Net Sum	Target	Actual	Alpha*	Weight Values		
x1	x2	1.0*w0	x1*w1	x2*w2	Input	Output	Output	Error	w0	w1	w2
						t(y)	o(model)	(t-o)α	0.1	0.1	0.1
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	0.10	0.10
0	1	-0.40	0.00	0.10	-0.30	0	0	0.00	-0.40	0.10	0.10
1	0	-0.40	0.10	0.00	-0.30	0	0	0.00	-0.40	0.10	0.10
1	1	-0.40	0.10	0.10	-0.20	1	0	0.50	0.10	0.60	0.60
0	0	0.10	0.00	0.00	0.10	0	1	-0.50	-0.40	0.60	0.60
0	1	-0.40	0.00	0.60	0.20	0	1	-0.50	-0.90	0.60	0.10
1	0	-0.90	0.60	0.00	-0.30	0	0	0.00	-0.90	0.60	0.10
1	1	-0.90	0.60	0.10	-0.20	1	0	0.50	-0.40	1.10	0.60
0	0	-0.40	0.00	0.00	-0.40	0	0	0.00	-0.40	1.10	0.60
0	1	-0.40	0.00	0.60	0.20	0	1	-0.50	-0.90	1.10	0.10
1	0	-0.90	1.10	0.00	0.20	0	1	-0.50	-1.40	0.60	0.10
1	1	-1.40	0.60	0.10	-0.70	1	0	0.50	-0.90	1.10	0.60
0	0	-0.90	0.00	0.00	-0.90	0	0	0.00	-0.90	1.10	0.60
0	1	-0.90	0.00	0.60	-0.30	0	0	0.00	-0.90	1.10	0.60
1	0	-0.90	1.10	0.00	0.20	0	1	-0.50	-1.40	0.60	0.60
1	1	-1.40	0.60	0.60	-0.20	1	0	0.50	-0.90	1.10	1.10
0	0	-0.90	0.00	0.00	-0.90	0	0	0.00	-0.90	1.10	1.10
0	1	-0.90	0.00	1.10	0.20	0	1	-0.50	-1.40	1.10	0.60
1	0	-1.40	1.10	0.00	-0.30	0	0	0.00	-1.40	1.10	0.60
1	1	-1.40	1.10	0.60	0.30	1	1	0.00	-1.40	1.10	0.60
0	0	-1.40	0.00	0.00	-1.40	0	0	0.00	-1.40	1.10	0.60
0	1	-1.40	0.00	0.60	-0.80	0	0	0.00	-1.40	1.10	0.60
1	0	-1.40	1.10	0.00	-0.30	0	0	0.00	-1.40	1.10	0.60
1	1	-1.40	1.10	0.60	0.30	1	1	0.00	-1.40	1.10	0.60

Initialize weight



หยุดเมื่อ

1. ตอบถูกหมดแล้ว
2. Epoch หมด
3. Weight แกว่งไปมาน้อย

ขั้นตอนแรกเริ่มจากการสุ่มค่า w_0 จนถึง w_2 ในที่นี้กำหนดให้เป็น 0.1 ทั้งสามตัว จากนั้นก็เริ่มป้อนตัวอย่างเข้าไป (ทีละแถว) ตัวอย่างแรกได้ผลรวมเชิงเส้น (Net Sum) เป็น 0.10 ซึ่งมากกว่า 0 ดังนั้นเปอร์เซปตรอนจะให้เอาต์พุตจริง (Actual Output) ออกมาเป็น 1 ซึ่งผิดเพราะเอาต์พุตเป้าหมาย (Target Output) จะต้องได้เป็น 0 ทำให้อัตราการเรียนรู้คูณค่าผิดพลาด (Alpha x Error) ได้ –0.50 หลังจากนี้ก็นำไปปรับน้ำหนักตาม $w_i \leftarrow w_i + \Delta w_i$ และ $\Delta w_i \leftarrow \alpha(t-o)x_i$ ดังนั้นจะได้เป็น $w_0 \leftarrow w_0 + \alpha(t-o)x_0$ = w_0 + $\overset{\alpha}{\boxed{0.50(-1)}}$ x $\overset{x_0}{1}$ = 0.10 + (-0.5) = -0.4 ต่อไปก็ปรับค่า w_1 ในทำนองเดียวกัน $w_1 \leftarrow w_1 + \alpha(t-o)x_1$ = w_1 + 0.50(-1) x 0 ดังนั้น w_1 จะเท่ากับ 0.10 คือไม่เปลี่ยนแปลง เช่นเดียวกับ w_2 ที่ไม่เปลี่ยนแปลง จะเห็นได้ว่าแม้มีค่าผิดพลาดแต่ไม่มีการปรับค่า w_1 และ w_2 เนื่องจากอินพุตที่ใส่เข้าไปเป็น 0 ทำ

ตัวแบบฝึกหัด (A and B) or C

x_1	x_2	x_3	t (y)
1	1	0	1
0	1	0	0
0	0	1	1
1	1	1	1

TP = 1 (t = 1, O = 1) FP = 0

TN = 1 (t = 0, O = 0) FN = 0

Acc $\frac{TP+TN}{TP+TN+FP+FN} = \frac{1+1}{1+1+0+0} = 1.0$ (100%)

Precision $\frac{TP}{TP+FP} = \frac{1}{1+0} = 1.0$ (100%)

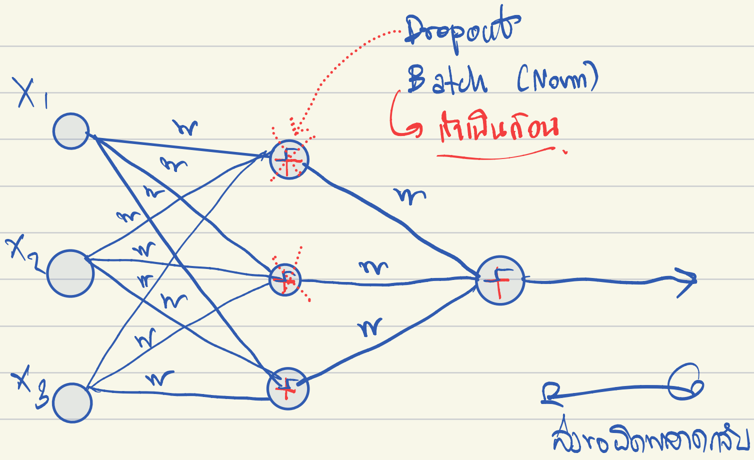
Recall $\frac{TP}{TP+FN} = \frac{1}{1+0} = 1.0$ (100%)

Space $\frac{TN}{TN+FP} = \frac{1}{1+0} = 1.0$ (100%)

5 Epoch

Bias Input $x_0 = +1$ Alpha = 0.1 Weight Values

	Input x_1	Input x_2	Input x_3	$1.0 * w_0$	$x_1 * w_1$	$x_2 * w_2$	$x_3 * w_3$	Net Sum Input	t Output	O (model) Output	Alpha Error	w_0	w_1	w_2	w_3
												0.2	0.2	0.2	0.2
Epoch1	1	1	0	0.2	0.2	0.2	0	0.6	1	1	0	0.2	0.2	0.2	0.2
	0	1	0	0.2	0	0.2	0	0.4	0	1	-0.1	0.1	0.2	0.1	0.2
	0	0	1	0.1	0	0	0.2	0.3	1	1	0	0.1	0.2	0.1	0.2
	1	1	1	0.1	0.2	0.1	0.2	0.6	1	1	0	0.1	0.2	0.1	0.2
Epoch2	1	1	0	0.1	0.2	0.1	0	0.4	1	1	0	0.1	0.2	0.1	0.2
	0	1	0	0.1	0	0.1	0	0.2	0	1	-0.1	0	0.2	0	0.2
	0	0	1	0	0	0	0.2	0.2	1	1	0	0	0.2	0	0.2
	1	1	1	0	0.2	0	0.2	0.4	1	1	0	0	0.2	0	0.2
Epoch3	1	1	0	0	0.2	0	0	0.2	1	1	0	0	0.2	0	0.2
	0	1	0	0	0	0	0	0	0	1	-0.1	-0.1	0.2	-0.1	0.2
	0	0	1	-0.1	0	0	0.2	0.1	1	1	0	-0.1	0.2	-0.1	0.2
	1	1	1	-0.1	0.2	-0.1	0.2	0.2	1	1	0	-0.1	0.2	-0.1	0.2
Epoch4	1	1	0	-0.1	0.2	-0.1	0	0	1	1	0	-0.1	0.2	-0.1	0.2
	0	1	0	-0.1	0	-0.1	0	-0.2	0	0	0	-0.1	0.2	-0.1	0.2
	0	0	1	-0.1	0	0	0.2	0.1	1	1	0	-0.1	0.2	-0.1	0.2
	1	1	1	-0.1	0.2	-0.1	0.2	0	1	1	0	-0.1	0.2	-0.1	0.2
Epoch5	1	1	0	-0.1	0.2	-0.1	0	0	1	1	0	-0.1	0.2	-0.1	0.2
	0	1	0	-0.1	0	-0.1	0	-0.2	0	0	0	-0.1	0.2	-0.1	0.2
	0	0	1	-0.1	0	0	0.2	0.1	1	1	0	-0.1	0.2	-0.1	0.2
	1	1	1	-0.1	0.2	-0.1	0.2	0	1	1	0	-0.1	0.2	-0.1	0.2
Test	1	0	1	-0.1	0.2	0	0.2	0.3	1	1	0	-0.1	0.2	-0.1	0.2
	0	0	0	-0.1	0	0	0	-0.1	0	0	0	-0.1	0.2	-0.1	0.2
Add	0	1	1	-0.1	0	-0.1	0.2	0	1	1	0	-0.1	0.2	-0.1	0.2
	1	0	1	-0.1	0.2	0	0.2	0.4	1	1	0	-0.1	0.2	-0.1	0.2
Test2	1	0	1	-0.1	0.2	0	0.2	0.3	1	1	0	-0.1	0.2	-0.1	0.2
	0	0	0	-0.1	0	0	0	-0.1	0	0	0	-0.1	0.2	-0.1	0.2


Dropout
Batch (Norm)
ทำเป็นก้อน
X_1
x_2
x_3
w
f
R
0
ส่งค่าผิดพลาดกลับ